Step 2 — That Specialist, Now a Real Agent That Can Improve Its Own Work — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: a first draft is rarely the best one. This step gives the Writer a way to improve: a Reviewer agent scores each draft against written criteria, and a small `while` loop sends the feedback back to the Writer for another attempt — bounded by a hard `MAX_REVISIONS` limit so it can't run forever. It's still just two agents in a plain Python loop, with no graph yet — that comes in Step 5. **If not:** Step 5 would be the first place the revision loop ever ran, inside a much bigger system with a graph to debug alongside it.

After this step the project runs: it writes an article, has it reviewed, and revises it until it passes or the limit is reached. Files not shown here don't change.

Solution — a scored verdict, written criteria, and an honest report

Changed from Step 1: `writer.txt` gets a feedback section; `run_writer()` gets a `feedback` parameter.

Story — `prompts/writer.txt` + `agents/writer_agent.py`: the chain itself is unchanged; it now accepts a `feedback` field so it can be told specifically what to fix on a revision. An empty feedback (the first draft, with nothing to revise yet) becomes the literal word "none", so the prompt always reads as a complete sentence instead of leaving a blank. **If not:** the Writer would redraft blindly, and could repeat the exact same mistake every round.

`src/prompts/writer.txt`

```
You are the Writer on a content team. Write a short article (150-250
words) with a title, using ONLY the findings below. Do not add facts
that are not in the findings. Keep each fact's [source] tag.

Findings:
{findings}

Reviewer feedback on your last draft (fix every point, or "none"):
{feedback}
```

```
# src/agents/writer_agent.py
# The Writer agent. build_writer_chain() is the LCEL chain; run_writer()
# turns findings (and optional feedback) into a draft.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.models.llm import get_model
from src.utils.prompts import load_prompt

def build_writer_chain():
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
    return prompt | get_model() | StrOutputParser()

# changed in Step 2: + feedback
def run_writer(findings, feedback=""):
    if feedback == "":
        feedback = "none"
    chain = build_writer_chain()
    return chain.invoke({"findings": findings, "feedback": feedback})
```

Story — `schemas.py`: this defines the Reviewer's verdict as a checked Pydantic object — a score from 1 to 10 and the feedback text — returned through `with_structured_output` (Doc05's pattern) instead of free text. **If not:** you'd be parsing something like "GOOD"/"BAD" out of prose, hoping the model's wording never drifts.

```
# src/schemas.py
# ReviewVerdict: the Reviewer's reply as a checked object — a score (1-10)
# the code compares to the pass mark, and the feedback text.
from pydantic import BaseModel, Field

class ReviewVerdict(BaseModel):
    score: int = Field(ge=1, le=10)
    feedback: str
```

Story — `prompts/reviewer.txt`: this spells out three concrete criteria — clarity, grounding in the findings, completeness — and the explicit rule "score 7 or higher only if all three are genuinely met." Without named criteria a model has nothing specific to judge against. **If not:** a vague Reviewer approves the first draft almost every time, and the revision loop never actually gets exercised.

`src/prompts/reviewer.txt`

```
You are the Reviewer on a content team. Score this draft from 1 to 10
against three criteria:
1. Clarity - is it easy to read, with a clear title and structure?
```

```
2. Grounding - is every fact backed by a [source] tag from the findings?
3. Completeness - does it cover the main findings, in 150-250 words?
Give 7 or more only if all three criteria are met. In the feedback, list
exactly what to fix.

Findings the draft must be based on:
{findings}

Draft:
{draft}
```

Story — `agents/reviewer_agent.py`: the model's job is only to produce a score; deciding what counts as "passing" is **plain Python** (`PASS_SCORE = 7`), not something the model decides for itself. Keeping the threshold in code means it's one constant you can see, test, and change in one place. **If not:** "pass" would drift with the model's mood from run to run, with no fixed line anyone could point to.

```
# src/agents/reviewer_agent.py
# The Reviewer agent. PASS_SCORE is the pass mark, decided in code.
# run_reviewer(findings, draft) returns a checked ReviewVerdict.
from langchain_core.prompts import ChatPromptTemplate

from src.models.llm import get_model
from src.schemas import ReviewVerdict
from src.utils.prompts import load_prompt

PASS_SCORE = 7

def run_reviewer(findings, draft):
    prompt = ChatPromptTemplate.from_template(load_prompt("reviewer"))
    chain = prompt | get_model().with_structured_output(ReviewVerdict)
    return chain.invoke({"findings": findings, "draft": draft})
```

Story — `main.py`: `write_with_review()` is the Generator → Critic loop (Doc11's pattern): write a draft, review it, if rejected pass the feedback back to the Writer, repeat until accepted or `MAX_REVISIONS` is hit. It returns a dict with the **same keys the Step 5 team will use** (`draft` , `accepted` , `feedback_history` , `revision_count`), and `print_report(result)` prints it, carefully never showing a rejected draft as if it were finished. Keeping `print_report()` 's name and input shape stable is deliberate — the rest of the project builds on calling it the same way. **If not:** a Writer and Reviewer that never agree would loop forever, and Step 5 would have to rewrite how results are printed.

```
# main.py
# The file you run: python main.py
```

```
# Step 2 version: write_with_review() loops the Writer and Reviewer up to
# MAX_REVISIONS; print_report() shows the result honestly.
import sys

from src.agents.reviewer_agent import PASS_SCORE, run_reviewer
from src.agents.writer_agent import run_writer
from src.config import load_config
from src.exceptions import MissingConfigError
from src.utils.logger import get_logger

MAX_REVISIONS = 3

SAMPLE_FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)

def write_with_review(findings):
    logger = get_logger(__name__)
    result = {
        "draft": "",
        "accepted": False,
        "feedback_history": [],
        "revision_count": 0,
    }
    feedback = ""
    for round_number in range(1, MAX_REVISIONS + 1):
        result["draft"] = run_writer(findings, feedback)
        verdict = run_reviewer(findings, result["draft"])
        logger.info(f"round {round_number}: score {verdict.score}")
        if verdict.score >= PASS_SCORE:
            result["accepted"] = True
            return result
        feedback = verdict.feedback
        result["feedback_history"].append(feedback)
        result["revision_count"] = result["revision_count"] + 1
    return result

def print_report(result):
    if result["accepted"]:
        print("\nFinal article:\n")
        print(result["draft"])
    else:
        attempts = result["revision_count"]
        print(f"\nCouldn't agree after {attempts} revisions. Feedback:")
        number = 1
        for feedback in result["feedback_history"]:
```

```
            print(f"  round {number}: {feedback}")
            number = number + 1
        print("\nLast draft (NOT approved):\n")
        print(result["draft"])

def main():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    result = write_with_review(SAMPLE_FINDINGS)
    print_report(result)

if __name__ == "__main__":
    main()
```

Story — `tests/integration/test_live_agents.py`: this adds the one check that matters most for this step: a deliberately weak, thin draft must **fail** the Reviewer's checks (case 2), against the real model. A Reviewer that approves good drafts proves little on its own. **If not:** a Reviewer that passes everything looks exactly like one that actually works.

```
# tests/integration/test_live_agents.py
# Runs the agents against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
from src.agents.reviewer_agent import PASS_SCORE, run_reviewer
from src.agents.writer_agent import run_writer
from src.config import load_config

FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)


def test_writer_keeps_sources():
    draft = run_writer(FINDINGS)
    result = "OK"
    if "[" not in draft:
        result = "WRONG"
    print(f"case 1 (writer keeps sources) -> {result}")


def test_reviewer_rejects_weak_draft():
```

```
    verdict = run_reviewer(FINDINGS, "E-bikes are good.")
    result = "OK"
    if verdict.score >= PASS_SCORE:
        result = "WRONG"
    print(f"case 2 (weak draft fails)     -> {result}")


if __name__ == "__main__":
    load_config()
    test_writer_keeps_sources()
    test_reviewer_rejects_weak_draft()
```

Run it (from the repo root):

```
python -m tests.integration.test_live_agents
python main.py
```

Expected output:

```
case 1 (writer keeps sources) -> OK
case 2 (weak draft fails)     -> OK
```

To see the limit, set `PASS_SCORE = 11` for one run: you should get "Couldn't agree after 3 revisions" and the full feedback list — never an endless loop.